CUDA-Accelerated Smith–Waterman Alignment of Synthetic DNA Sequences

Antonio Nikoloski
Skopje, North Macedonia
Email: antonionikoloski2003@gmail.com

***Abstract*—Local sequence alignment dominates the cost of many genomic analysis pipelines, with the Smith–Waterman algorithm representing a major bottleneck due to its quadratic complexity. This work presents a CUDA implementation that maps independent alignments to individual GPU threads using a memory-efficient dynamic programming strategy. We benchmark it against serial and multiprocessing CPU implementations on reference–query pairs sampled from the human chromosome 1 reference (GRCh38), where each query carries simulated, transition-biased substitutions that mimic the transition excess observed in cancer genomes. For sequences of length 128 and up to 6,000 alignments, the GPU is between 3,500× and 4,400× faster than serial CPU execution and between 1,100× and 2,800× faster than multiprocessing CPU execution, while producing the same scores as the CPU implementation on the validation set. The mutations are synthetic, so this work is a performance and correctness benchmark rather than a clinical variant detector.**

***Index Terms*—GPU computing, CUDA, Smith-Waterman alignment, synthetic mutations, parallel bioinformatics, benchmarking**

I. Introduction

Whole-genome sequencing (WGS) plays a central role in modern cancer genomics by allowing the identification of somatic mutations that drive tumor development and progression [15]. Large-scale initiatives such as The Cancer Genome Atlas (TCGA) have demonstrated that accurate detection of single-nucleotide variants (SNVs) and small insertions or deletions is critical to understanding cancer mechanisms [15].

A major computational challenge in WGS analysis arises from sequence alignment, which constitutes a dominant portion of the total runtime in genomic pipelines [11]. Classical CPU-based tools, including BWA-MEM and GATK-derived workflows, provide high accuracy but often require substantial execution time when applied to whole-genome or tumor-normal datasets [4], [11]. As sequencing throughput continues to increase, these CPU-only approaches struggle to satisfy performance and scalability requirements.

Graphics Processing Units (GPUs) have emerged as an effective platform for accelerating data-parallel workloads in bioinformatics due to their ability to execute thousands of concurrent threads [1], [2]. Previous studies have shown that dynamic programming algorithms, such as Smith-Waterman (SW) local alignment, are particularly well suited for GPU acceleration [1], [2]. Despite its biological sensitivity and accuracy, the quadratic time complexity of SW makes naive CPU implementations impractical for large-scale mutation analysis [1].

GPU-based acceleration of SW alignment has therefore become a key research direction in high-performance genomics. Existing work demonstrates that fine-grained parallelization can substantially reduce alignment time while preserving alignment correctness [1]– [3].

In this work, we present a CUDA implementation of Smith-Waterman alignment together with a reproducible benchmark. Our approach adopts a heterogeneous CPU-GPU architecture, where the CPU performs genome preprocessing, synthetic mutation generation and result aggregation, while the GPU executes massively parallel alignment kernels. Queries are generated with transition-biased substitutions, a simple model inspired by the excess of transitions such as C→T observed in cancer genomes. We compare the runtime of serial CPU, multiprocessing CPU and GPU execution and verify that all implementations produce the same scores. Because the mutations are simulated, the results characterize performance and score behavior, not clinical detection accuracy.

II. Related Work

Research on accelerating local sequence alignment has produced a broad range of GPU-based solutions, which can be broadly categorized into alignment libraries, integrated genomic frameworks, and mutation-aware analysis systems.

A. GPU-Accelerated Smith–Waterman Alignment

Numerous studies have focused on accelerating the Smith-Waterman algorithm using GPU architectures. CUDASW++4.0 demonstrates that high performance can be achieved through wavefront parallelization, optimized memory access, and warp-level primitives, enabling ultra-fast database-scale alignments [1]. Similarly, GASAL2 provides flexible GPU kernels optimized for high-throughput short-read alignment, emphasizing asynchronous execution and memory efficiency [2]. AnySeq/GPU further explores alternative data layouts and kernel designs to reduce memory bottlenecks during alignment [3]. Collectively, these works establish GPU-based Smith–Waterman alignment as a mature and effective approach for high-performance sequence comparison.

B. GPU-Accelerated Genomic Pipelines

Beyond standalone alignment kernels, several frameworks integrate GPU acceleration into complete genomic analysis pipelines. SOAP3-dp and BarraCUDA were among the earliest systems to demonstrate that short-read alignment based on

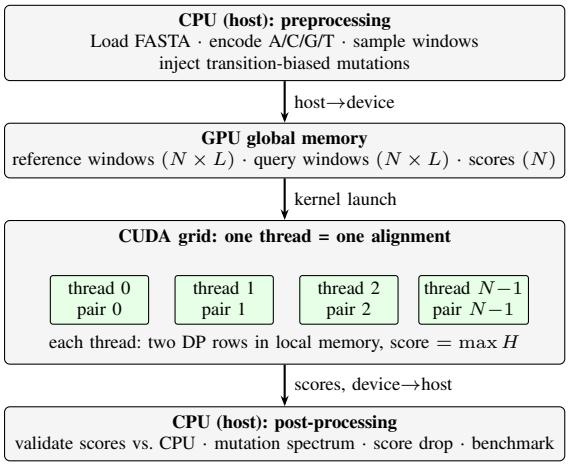


Fig. 1. Overall system architecture. The CPU handles genome preprocessing, synthetic mutation generation and result aggregation, while the GPU runs one independent Smith–Waterman alignment per thread.

Burrows-Wheeler indexing could be efficiently implemented on GPUs [6], [7]. More recently, NVIDIA Parabricks and the NVBIO library have shown that GPU acceleration can significantly reduce end-to-end execution time of industry-standard genomic workflows, including variant calling pipelines derived from GATK [4], [5]. Benchmarking studies confirm that such GPU-accelerated pipelines offer both performance and energy-efficiency advantages over CPU-only solutions [11].

C. Pre-alignment Filtering and Variant Calling

To further reduce the computational cost of alignment, pre-alignment filtering techniques have been proposed. SneakySnake introduces a universal filtering framework that discards unlikely alignment candidates before invoking expensive Smith-Waterman computations, achieving significant speedups across CPU, GPU, and FPGA platforms [10]. In parallel, modern variant calling systems increasingly incorporate machine learning techniques. Tools such as DeepVariant and Clair3 leverage neural networks to improve SNP and indel detection accuracy, often relying on GPU acceleration during inference [8], [9]. While these approaches enhance variant calling accuracy, they still depend on efficient alignment as a prerequisite.

In contrast to end-to-end frameworks, our work focuses on the Smith-Waterman alignment kernel itself and on a reproducible benchmark with controlled, transition-biased synthetic mutations. It does not attempt variant calling.

III. Solution Architecture and Implementation

This section presents the architecture and implementation details of the proposed CUDA-accelerated Smith-Waterman (SW) framework. The focus is placed on how genomic data are processed, mapped to GPU threads, and analyzed to characterize the effect of transition-biased synthetic mutations on alignment scores.

A. Overall System Architecture

The system adopts a heterogeneous CPU–GPU execution model. The CPU handles genome preprocessing, mutation creation, and consolidation of the results, whereas the GPU carries out highly parallel local sequence alignments using the Smith–Waterman algorithm.

The workflow consists of the following stages:

1) **Genome Encoding:** Input DNA sequences from FASTA files are encoded into integer representations ($A = 0, C = 1, G = 2, T = 3$). Ambiguous bases split a sequence into separate contiguous segments.
2) **Window Extraction:** Fixed-length genomic windows are sampled uniformly from the valid segments of the reference genome.
3) **Synthetic Mutation Injection:** Query sequences are generated by mutating each base of a reference window independently with a configurable probability. A mutation is a transition ($A \leftrightarrow G$, $C \leftrightarrow T$) with probability 0.7 and a transversion otherwise.
4) **Parallel Alignment:** Each reference-query pair is aligned independently on the GPU using the Smith-Waterman algorithm.
5) **Post-processing:** Alignment scores are transferred back to the CPU for validation, mutation statistics and benchmarking.

This design ensures minimal data dependency between alignments, making the problem well-suited for GPU parallelization.

B. GPU Thread and Memory Mapping

Each alignment pair is mapped to a single CUDA thread, allowing thousands of independent Smith-Waterman computations to be executed concurrently. The mapping strategy is illustrated in Fig. 1.

The memory usage of the kernel is organized as follows:

- **Global Memory:** Stores the encoded reference windows, the query sequences and the output scores.
- **Thread-local arrays:** Each thread keeps only two rows of the dynamic programming matrix (two arrays of 1,025 integers, supporting $L \leq 1024$) instead of the full matrix, so memory grows linearly with L rather than quadratically. Because these arrays are indexed dynamically, they reside in local memory rather than registers.
- **Scoring parameters:** The match, mismatch and gap scores are compile-time constants; shared memory is not used.

Keeping two rows removes the quadratic memory requirement, but the per-thread footprint of about 8 KB limits occupancy, which is consistent with the throughput plateau observed in Section IV.

TABLE I
SIMPLIFIED SMITH–WATERMAN SCORING MATRIX

	$*$	A	C	G
$*$	0	0	0	0
A	0	2	1	0
T	0	1	1	0
G	0	0	0	3

C. Smith-Waterman Recurrence Relation

The Smith-Waterman algorithm computes a local alignment score matrix H using the following recurrence relation:

$$H(i,j) = \max \begin{cases} 0, \\ H(i-1,j-1) + s(x_i, y_j), \\ H(i-1,j) + g, \\ H(i,j-1) + g \end{cases}$$

where x_i and y_j represent nucleotides from the query and reference sequences, respectively, $s(\cdot)$ is the substitution score ($+2$ for a match, -1 for a mismatch), and $g = -1$ is the gap penalty. The maximum value in H corresponds to the optimal local alignment score $H_{\max}$.

In the proposed implementation, each CUDA thread computes this recurrence independently for its assigned sequence pair.

D. Concrete Alignment Example

To clarify the alignment process, consider a simplified example with a reference sequence

$$R = \{A, C, G\}$$

and a query sequence containing a transition mutation:

$$Q = \{A, T, G\}$$

Using a match score of $+2$, mismatch penalty of -1, and gap penalty of -1, the resulting Smith–Waterman scoring matrix is shown in Table I.

The mismatch between C (reference) and T (query) reduces the score but preserves a valid local alignment path. This illustrates how a transition mutation lowers the score without destroying the similarity between the two sequences.

E. Mutation and Score Analysis

After computing $H_{\max}$ for each alignment, the following analysis is applied:

1) **Mutation Spectrum:** The system counts the injected transitions ($A \leftrightarrow G$, $C \leftrightarrow T$) and transversions to verify the configured transition bias.
2) **Score Drop:** For each pair, the score of the reference aligned with itself is compared with the score of the mutated query, and the difference is the score drop.
3) **Statistical Aggregation:** Counts and score drops are aggregated on the CPU for benchmarking and mutation impact analysis.

TABLE II
RUNTIME PERFORMANCE COMPARISON (SECONDS, $L = 128$)

N (Alignments)	CPU Serial (s)	CPU Parallel (s)	GPU (s)
500	20.7	13.1	0.0047
1000	41.5	19.0	0.0096
2000	83.5	31.1	0.0234
3000	124.3	44.0	0.0347
4000	165.7	54.7	0.0472
5000	207.7	66.8	0.0585
6000	249.4	80.0	0.0679

IV. EXPERIMENTAL RESULTS AND DISCUSSION

In this section, we present a quantitative performance assessment of the implemented Smith–Waterman algorithm under three execution paradigms: CPU serial execution, CPU parallel execution via multiprocessing, and GPU-accelerated execution using CUDA. We also verify that the GPU scores agree with the CPU scores and analyze the degradation of alignment scores as a function of controlled mutation rates.

A. Runtime Performance and Scalability

The execution time was measured for a varying number of sequence pairs (N), ranging from 500 to 6,000. Each pair consisted of a reference window from human chromosome 1 (GRCh38) and a query of length $L = 128$ generated with a mutation rate of 0.10 and a random seed of 42. Each configuration was repeated five times and the best time is reported, which excludes one-off warm-up and system interruptions. GPU timings include memory allocation and host–device transfers, and CPU parallel timings include process-pool creation. The results are summarized in Table II and visualized in Fig. 2.

Table II illustrates a distinct difference in scalability between the CPU and GPU implementations:

1) **CPU Serial:** The serial CPU implementation exhibits a linear increase in execution time with respect to the input size ($O(N)$), at about 41.5 seconds per 1,000 alignments, requiring 249.4 seconds for $N = 6{,}000$.
2) **CPU Parallel:** The parallel CPU implementation distributes the alignments over multiple processes. Although it is faster than the serial version (80.0 seconds for $N = 6{,}000$), it is only about $3.1\times$ faster at this size, so the scaling with the number of cores is sublinear. Each call also pays a fixed start-up cost of about 7 seconds for process creation, in addition to inter-process communication (IPC) overhead in Python.
3) **GPU CUDA:** The CUDA implementation completes $N = 6{,}000$ alignments in 0.068 seconds. GPU time grows approximately linearly with N (0.0047 s at $N = 500$), corresponding to a throughput of roughly 85,000 to 106,000 alignments per second, so the device is already saturated at small N rather than reaching a plateau only for large N.

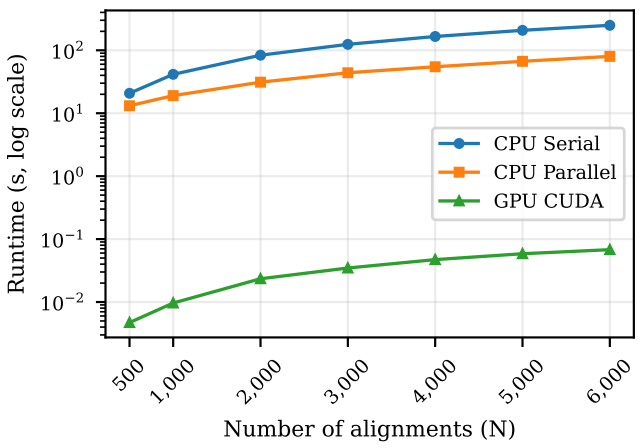


Fig. 2. Runtime scaling of CPU Serial, CPU Parallel and GPU CUDA implementations ($L = 128$, logarithmic runtime axis).

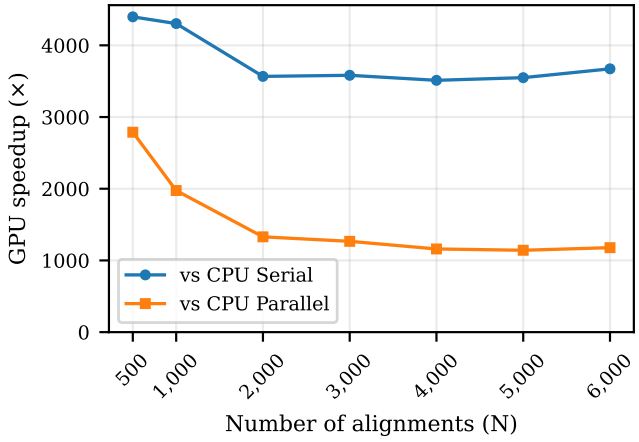


Fig. 3. GPU speedup relative to CPU Serial and CPU Parallel execution ($L = 128$).

B. Acceleration and Speedup Analysis

The principal metric used to assess the efficiency of the GPU implementation is the speedup factor, defined as $S = T_{\text{CPU}}/T_{\text{GPU}}$. Figure 3 and Table III report the speedup obtained by the GPU relative to both the serial and the parallel CPU implementations.

The results indicate a substantial performance improvement. For $N = 6{,}000$, the GPU achieves a speedup of approximately $3{,}672\times$ over the serial CPU implementation and $1{,}178\times$ over the parallel CPU version. Over the whole range, the speedup is between $3{,}512\times$ and $4{,}397\times$ against the serial CPU and between $1{,}142\times$ and $2{,}788\times$ against the parallel CPU. The speedup is highest for the smallest workloads and settles near $3{,}500\times$ (serial) and $1{,}150\times$ (parallel) once the GPU is saturated. The speedup against the parallel CPU is inflated at small N by its fixed start-up cost.

TABLE III
GPU SPEEDUP FACTOR ($L = 128$)

N (Alignments)	Speedup vs. Serial	Speedup vs. Parallel
500	4,397×	2,788×
1000	4,303×	1,975×
2000	3,567×	1,329×
3000	3,582×	1,266×
4000	3,512×	1,160×
5000	3,549×	1,142×
6000	3,672×	1,178×

TABLE IV
IMPACT OF MUTATION RATE ON ALIGNMENT SCORE

Mutation Rate	Mean Score Drop	Std. Deviation
0.00	2.98	0.13
0.10	36.72	9.40
0.30	101.16	12.57
0.50	147.49	11.31
0.80	174.37	12.55
1.00	180.76	16.17

C. Score Agreement and Mutation Analysis

To ensure the GPU implementation is correct, its scores were compared with those of the CPU implementation. Both produced identical scores on 64 sequence pairs (0 mismatches), and boundary sequence lengths of 1, 128 and 1,024 passed. The mutation generator was validated as well: for a target mutation rate of 10% the observed rate was 9.93%, and for a target transition fraction of 70% the observed fraction was 69.69%.

We then analyzed the impact of mutation rates on the alignment score, testing rates from 0.0 (identical sequences) to 1.0 (every position selected for mutation). As shown in Table IV and Fig. 4, there is a strong inverse relationship between the mutation rate and the alignment score. The mutation-rate experiment was run with an earlier version of the generator that forced at least one mutation in every window.

- At a **0.0 mutation rate**, the mean score drop is small ($\approx$ 2.98) because of that single forced mutation: one mismatch in the middle of a perfect 256-point alignment costs about 3 points.
- As the rate increases to **0.5**, the mean score drop increases to 147.49, showing that the scoring correctly penalizes mismatches and gaps.
- At **1.0**, the score drop saturates around 180, indicating that little local similarity remains.

This behavior shows that the alignment scores respond monotonically to the amount of simulated mutation, while the identical CPU and GPU scores confirm that the acceleration does not change the result. These checks concern the implementation; they do not establish clinical sensitivity or specificity.

V. CONCLUSION

In this work, we developed and evaluated a CUDA implementation of the Smith-Waterman local alignment algorithm

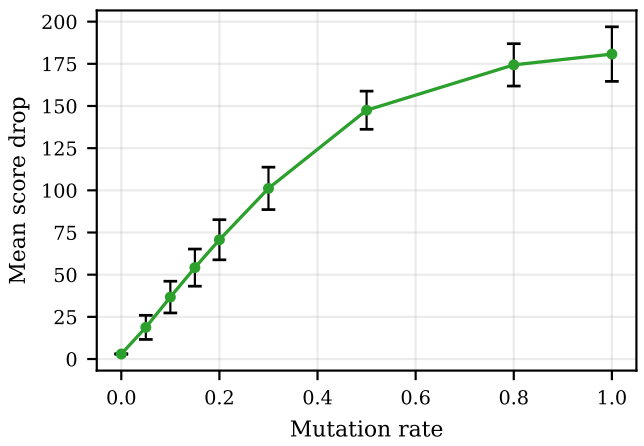


Fig. 4. Impact of mutation rate on alignment score drop (mean and standard deviation).

and benchmarked it on synthetic, transition-biased mutations of human reference sequences. By mapping one alignment to each GPU thread, we reduced the cost of quadratic-time dynamic programming for batches of short sequences.

For sequences of length 128 and up to 6,000 alignments, the GPU implementation was between $3{,}500\times$ and $4{,}400\times$ faster than serial CPU execution and between $1{,}100\times$ and $2{,}800\times$ faster than multiprocessing CPU execution, at a throughput of roughly 85,000 alignments per second, and produced the same scores as the CPU implementations on the validation set.

The study has clear limits. The mutations are simulated substitutions without insertions or deletions, the transition bias is a model parameter, the kernel supports sequences of up to 1,024 bases, and the measurements depend on the hardware used. Future work includes reducing the per-thread memory footprint of the kernel, reusing the CPU process pool, and extending the benchmark to realistic variant data.

References

[1] B. Schmidt, J. Haas, and M. Sanders, "CUDASW++4.0: Ultra-fast GPU-based Smith–Waterman protein sequence database search," *BMC Bioinformatics*, vol. 25, no. 1, pp. 1–14, 2024.

[2] N. Ahmed, A. Alser, and M. Alser, "GASAL2: A GPU-accelerated sequence alignment library for high-throughput NGS data," *BMC Bioinformatics*, vol. 20, no. 1, pp. 1–12, 2019.

[3] A. Müller, T. Hansen, and S. Klein, "AnySeq/GPU: A novel approach for faster sequence alignment on GPUs," arXiv preprint arXiv:2203.01234, 2022.

[4] J. O'Connell, R. Schneider, and NVIDIA Parabricks Team, "Accelerating genomic workflows using NVIDIA Parabricks," *BMC Bioinformatics*, vol. 24, no. 1, pp. 1–10, 2023.

[5] NVIDIA NVLabs, "NVBIO: A GPU-accelerated bioinformatics library," GitHub Repository, 2024. [Online]. Available: https://github.com/NVlabs/nvbio

[6] R. Luo, B. Liu, Y. Xie, Z. Li, and W. Huang, "SOAP3-dp: Fast, accurate and sensitive GPU-based short-read aligner," *Bioinformatics*, vol. 29, no. 1, pp. 13–20, 2013.

[7] P. Klus, S. Lam, D. Hamelryck, and J. S. Pedersen, "BarraCUDA: A fast short-read sequence aligner using GPUs," *Bioinformatics*, vol. 28, no. 22, pp. 2970–2971, 2012.

[8] R. Poplin et al., "DeepVariant: A universal SNP and small-indel variant caller using deep neural networks," *Nature Biotechnology*, vol. 36, no. 10, pp. 983–987, 2018.

[9] Z. Zheng, Y. Li, and H. Cheng, "Clair3: Efficient and accurate long-read variant calling," GitHub Repository, 2021. [Online]. Available: https://github.com/HKU-BAL/Clair3

[10] M. Alser, N. Ahmed, A. Alkan, and O. Mutlu, "SneakySnake: A fast and accurate universal genome pre-alignment filter for CPUs, GPUs, and FPGAs," arXiv preprint arXiv:1905.08430, 2019.

[11] N. Carpi, F. Cappello, and G. Bongiovanni, "Benchmarking GPU-accelerated genomics workflows," *Frontiers in Genetics*, vol. 13, pp. 1–14, 2022.

[12] SW-actors Consortium, "Recent Smith–Waterman GPU systems," Technical Report, 2023–2025.

[13] M. Steinegger, J. Söding, and T. Mirdita, "MMseqs2-GPU: High-performance sequence alignment on GPUs," *Bioinformatics*, vol. 40, no. 1, pp. 1–8, 2024.

[14] NVIDIA Developer Blog, "NVIDIA Clara and high-performance computing applications in genomics," NVIDIA Corporation, 2023.

[15] The Cancer Genome Atlas Research Network, "Comprehensive molecular portraits of human cancers," *Nature Genetics*, vol. 45, no. 10, pp. 1113–1120, 2013.